

Voices of Our Community

A live word cloud for a civic-engagement icebreaker.

Students scan a QR code, send **one word** for a problem in their community, and it lands on the projector instantly. When you unlock speaking, a student can tap **Speak** and their word turns blue and grows on the big screen, so the room knows who is about to talk.

Route	Who it's for	What it does
<code>/#/</code>	Students	Send one word. Tap Speak once you unlock it.
<code>/#/cloud</code>	The projector	The cloud, a live count, and the join QR code.
<code>/#/admin</code>	You	Sign in to unlock speaking, edit or delete words, open/close submissions.

Setup (about 10 minutes)

1. Create the Supabase project

1. Go to supabase.com → **New project**. Any region near you.
2. When it finishes provisioning, open **SQL Editor** → **New query**.
3. Paste the entire contents of [supabase/schema.sql](#) and hit **Run**. It creates the tables, locks them down with row-level security, and turns on realtime. Running it twice is safe.

2. Create your admin login

Authentication → **Users** → **Add user**

- Any email and password (it never sends mail).
- Tick **Auto Confirm User**, otherwise you cannot sign in.

That email and password is what you type on `/#/admin`. Students never sign in.

3. Point the app at your project

Project Settings → **API**, then copy the two values into a `.env` file next to `package.json`:

```
VITE_SUPABASE_URL=https://your-project-ref.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOi...
```

`VITE_EVENT_TITLE` is optional — it's the heading on the projector.

The anon key is meant to be public. It is safe in the browser: the SQL in step 1 restricts what an anonymous visitor can actually do (see [Security](#) below).

4. Run it

```
npm install
```

```
npm run dev
```

Open <http://localhost:5173/#/cloud> on the projector.

For phones to reach your laptop, they must be on the same wifi, and you need your laptop's LAN address rather than `localhost`. The dev server already listens on the network — look for the **Network:** line it prints, e.g. <http://192.168.1.20:5173>. The QR code on the projector encodes whatever address the browser is showing, so open the cloud page using that network address and the QR will be correct.

If the campus wifi blocks device-to-device traffic, deploy instead (below) — that always works and is the safer bet for a live session.

Deploy

Any static host works. The build is a plain SPA using hash routing, so no server-side rewrite rules are needed anywhere.

```
npm run build
```

Vercel / Netlify — connect the repo, framework "Vite", and add the two `VITE_...` environment variables in the dashboard.

GitHub Pages — a workflow is already included at <.github/workflows/deploy.yml>. In the repo:

1. **Settings** → **Secrets and variables** → **Actions** → **New repository secret** — add `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`.
2. **Settings** → **Pages** → **Source: GitHub Actions**.
3. Push to `main`. The site appears at <https://<user>.github.io/<repo>/#/>.

Running the activity

1. Open `/#/cloud` on the projector. Press **F** for fullscreen, **Q** to show/hide the QR panel.
2. Open `/#/admin` on your laptop (a second tab or your phone) and sign in.
3. Students scan and send their word. It appears within a second.
4. When you're ready to discuss, flip **Speaking enabled** on the admin page. The **Speak** button unlocks on every phone.
5. A student taps **Speak** — their word turns blue and grows. You'll see them listed under "hands raised".
6. Call on them. Afterwards hit **lower**, or **Clear all** to reset every hand.
7. Flip **Submissions open** off once everyone has contributed, so nobody adds words mid-discussion.

Useful details:

- **Identical words merge.** Three students typing "pollution" produce one big **Pollution³** rather than three overlapping copies — that's what makes the size differences meaningful. Any of those three raising a hand highlights it.
- **You can highlight a word yourself** from the admin word list ("highlight"), which is handy for pointing at a word while you talk about it.
- **Turning speaking off drops every raised hand** automatically.
- **One word per phone.** If a student needs theirs changed, edit or delete it in the admin list — deleting frees them to submit again.
- To reset between two classes, run **supabase/reset-session.sql**.

How the permissions work

Everything is enforced in Postgres, not in the browser, so a student poking at the console cannot get around it.

	Anonymous student	Signed-in admin
Read words and settings	yes	yes
Add a word	only while submissions are open	yes
Change a word's text	no	yes
Raise/lower own hand	only while speaking is enabled	yes
Delete a word	no	yes
Flip the switches	no	yes

The "change a word's text" row is a column-level grant, not just a policy:

```
revoke update on public.words from anon;  
grant update (isSpeaking) on public.words to anon;
```

So even during the window where students are allowed to update rows, the only column they can touch is **isSpeaking**. They cannot rewrite somebody else's word.

The one thing deliberately left open: a student could clear their browser storage to get a second submission, and hands are not tied to identity, so a student could lower another student's hand. Locking that down needs per-student accounts, which is not worth it for a classroom icebreaker — you can see and undo anything from the admin list.

Notes on the build

- **React + Vite**, no UI framework. **@supabase/supabase-js** and **qrcode** are the only runtime dependencies.
- **The cloud layout** (**src/lib/layout.js**) measures each word on a canvas, gives it a stable tilt seeded from the word itself (so it never jitters between renders, and never goes past $\pm 60^\circ$ or upside down),

then walks an elliptical spiral outward until it finds a gap. Sizes are decided up front and the whole ramp is scaled down together if the words would cover too much of the canvas, which keeps the frequency hierarchy intact instead of squashing whoever happens to be placed last. Verified to place 40 distinct words with zero overlap from 1024×560 up to 1920×1080.

- **Realtime plus a 4-second poll.** Realtime is what makes words appear instantly; the poll is a safety net so a dropped websocket on classroom wifi can't silently freeze the projector mid-session.
- **localStorage** holds the student's anonymous id and their word, so a refresh never restarts their turn. The server is still the source of truth — if you delete their word, their phone notices and lets them submit again.